Design and FPGA Implementation of Elliptic IIR Low-Pass Filter Architectures on the EBAZ4205

Le Vo Tien An, Nguyen Duy Hung, and Le The Dung
Faculty of Electrical and Electronics Engineering
Ho Chi Minh City University of Technology and Education
Ho Chi Minh City, Vietnam

***Abstract*—This paper presents the design and FPGA implementation of elliptic IIR low-pass filter architectures on the EBAZ4205 platform, which is based on the Zynq XC7Z010 device. Three hardware realizations are considered: a second-order DF1 single-biquad, a fourth-order DF1 cascade, and a fourth-order TDF2 cascade. The filters are designed in MATLAB, decomposed into second-order sections, quantized using Q2.28 coefficients, and implemented in Verilog. The end-to-end verification platform comprises an ESP32-based signal generator, an MCP4921 digital-to-analog converter, an MCP3208 analog-to-digital converter, FPGA filtering logic, and UART data transfer to a host computer. The experimental evaluation focuses on single-tone tests at 150 Hz and 250 Hz, representing passband and transition-band operation around the 200 Hz passband edge. An additional 350 Hz test is included only for the DF1 single biquad to illustrate the more gradual roll-off of the second-order. A 1.65 V DC input is also used to examine steady-state behavior and possible limit-cycle activity. Vivado implementation results show that all three architectures meet the 50 MHz clock constraint, while the measured resource usage reveals a trade-off among filter order, DSP usage, flip-flop usage, and internal register organization.**

***Index Terms*—elliptic IIR filter, FPGA implementation, fixed-point arithmetic, direct form I, transposed direct form II, digital filter design, embedded systems.**

I. Introduction

Digital signal processing plays an increasingly important role in communications, control, biomedical instrumentation, and embedded systems. Programmable hardware platforms such as FPGAs support high throughput, low latency, and parallel processing, making them suitable for real-time signal-processing applications [1], [2].

Among digital filter types, the Infinite Impulse Response filter is widely used because it can achieve a desired magnitude response with a lower filter order than an FIR filter. This advantage reduces the number of arithmetic operations and hardware resources. However, FPGA implementation of an IIR filter requires more than algorithm-level simulation. The design must also handle fixed-point coefficient quantization, sign extension, accumulator word length, overflow control, timing closure, and integration with peripheral hardware blocks such as ADC, DAC, SPI, and UART interfaces [3], [4].

Recent work on FPGA-based IIR filter implementation has explored several directions relevant to this study. Elliptic IIR structures have been applied to physiological-signal denoising, where a cascaded direct-form-II elliptic filter bank was realized on a Zynq-7000 ZedBoard using Xilinx System Generator to remove noise from ECG recordings [6]. Other studies have targeted arithmetic-level optimization of the IIR multiply-accumulate path itself, proposing improved adder structures to reduce area and improve timing of the filter's core datapath [7]; a more recent contribution along the same line combines a divide-and-conquer multiplier with multi-stage fast ripple hybrid adders to reduce the critical path and peak delay of the IIR datapath on FPGA [14]. A further line of work has explored FPGA realizations of higher-order elliptic IIR band-pass sections using graphical design tools, reporting on the achievable transition-band selectivity and resource cost of the elliptic structure on programmable logic [11]. Beyond the elliptic family, two additional recent directions are relevant here: one work imposes an explicit pole-radius stability constraint when synthesizing maximally flat IIR filters with low group delay, directly addressing the near-unit-circle pole sensitivity that also motivates the wide Q2.28 format adopted in this paper [13]; another describes a programmable, real-time FPGA filtering unit validated on live infrared-detector signals, which illustrates the practical value of closed-loop, signal-in/signal-out hardware verification for embedded filtering systems, but does so for a single filter realization rather than a structural comparison [12]. However, across this body of work, filters are generally evaluated either through post-synthesis simulation, XSG-level resource estimation, arithmetic-level synthesis metrics considered in isolation from a physical signal path, or a single hardware realization on an application-specific dataset; none of them combines a complete physical signal-generation-to-acquisition hardware loop with a direct, same-board comparison of multiple structural realizations of the identical transfer function. In this context, the contribution of this paper is threefold: (1) it presents a full end-to-end hardware verification path – from ESP32-based analog signal generation, through DAC/ADC conversion, FPGA filtering, and UART acquisition – rather than relying on simulation-only or estimator-based verification, extending the single-realization hardware-in-the-loop approach of [12] to a comparative setting; (2) it provides a direct, same-board comparison of DF1 single-biquad, DF1-cascade, and TDF2-cascade realizations of the same elliptic transfer function under an identical Q2.28 fixed-point format and identical 50 MHz timing constraint, isolating structural effects from coefficient or design differences; and (3) it quantifies not only frequency-domain accuracy but also DC steady-state stability and limit-cycle behavior arising from

the near-unit-circle pole radius – the same numerical-stability concern raised at the design-algorithm level in [13], but examined here empirically after fixed-point quantization and physical hardware implementation – together with resource utilization and post-route timing slack, within a single unified experimental platform on the low-cost EBAZ4205 board.

The remainder of this paper is organized as follows. Section II describes the system design and FPGA architectures. Section III presents the hardware verification results, DC stability, resource utilization, and timing analysis. Section IV concludes the paper.

II. System Design and Architecture

A. System Overview and Signal Path

The target platform is the EBAZ4205 board based on the Xilinx Zynq XC7Z010 [10]. The complete system consists of an ESP32 signal generator, an MCP4921 digital-to-analog converter, an MCP3208 analog-to-digital converter, the FPGA IIR filtering logic, and a USB-to-UART CP2102 interface for sending the filtered output to a host computer. Python captures the serial data, and MATLAB performs the subsequent waveform and spectral analysis. Fig. 1 illustrates the overall system architecture.

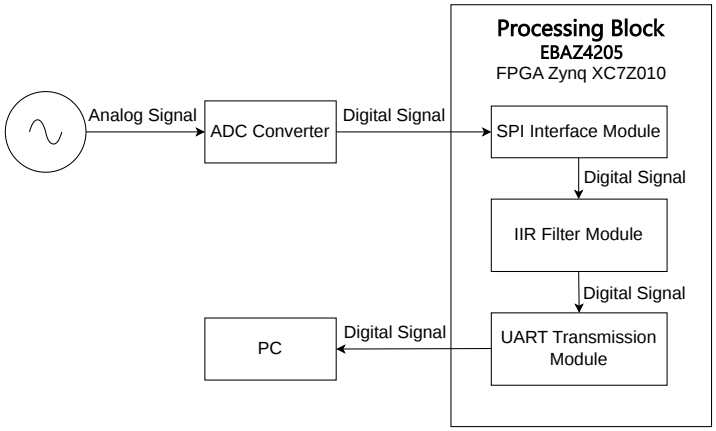


Fig. 1. System-level architecture: signal generation, digital-to-analog conversion, analog-to-digital conversion, FPGA-based IIR filtering, and UART transmission to the host PC.

The verification system follows an end-to-end signal path rather than evaluating the filter only in simulation. The input waveform is first generated as digital samples, converted to an analog voltage by the MCP4921 DAC, sampled again by the MCP3208 ADC, processed by the selected IIR filter architecture inside the FPGA, and finally transmitted to the host computer through UART. This path allows the hardware filter to be evaluated under practical conditions including DAC reconstruction error, ADC quantization, SPI timing, UART framing, and fixed-point arithmetic inside the FPGA [5].

The FPGA portion is organized as a single-clock synchronous design. The global clock of the EBAZ4205 board is 50 MHz. A divided-down SPI clock is used for the MCP3208 ADC, while the IIR filter core is enabled only when a new ADC sample is available. Thus, the filter states are not updated at every 50 MHz clock cycle, but only once per valid input sample. This is important for a recursive IIR filter because the feedback states must correspond exactly to the sampling instants of the ADC data stream.

TABLE I
Signal Path and Data Format Through the Proposed System

Stage	Signal Type	Data Format
Signal generator	Digital samples	12-bit unsigned
MCP4921 DAC	Analog voltage	0–3.3 V
MCP3208 ADC	Digital samples	12-bit unsigned
Offset removal	Signed samples	$x[n] - 2048$
FPGA filter core	Signed data	52-bit, 28 fractional bits
Filter coefficients	Fixed-point data	Q2.28 in 32-bit storage
Output saturation	Digital samples	12-bit unsigned
UART transmission	Serial frame	2 bytes/transmitted sample

The ADC sampling frequency is determined by the 50 MHz system clock, the SPI clock divider, and the number of SPI clock cycles required for one MCP3208 conversion. With a divider of 64 and 22 SPI clock cycles per conversion, the sampling frequency is

$$f_s = \frac{50 \times 10^6}{64 \times 22} \approx 35\,511 \text{ Hz}. \tag{1}$$

This value is used consistently in MATLAB filter design and FPGA hardware verification so that the frequency response measured in hardware corresponds to the theoretical response.

B. Elliptic Filter Design Methodology

Elliptic, or Cauer, filters are selected because they provide a sharper transition band than Butterworth and Chebyshev filters of the same order, at the cost of equiripple behavior in both the passband and the stopband [1]. This is suitable for low-order FPGA implementations where multiplier count, register count, and timing are important.

The design parameters are summarized in Table II. A second-order section is used as the DF1 single biquad, while the fourth-order filter is factored into two cascaded second-order sections for the DF1 cascade and TDF2 cascade realizations. Factoring the fourth-order transfer function into SOS form reduces coefficient sensitivity and facilitates section ordering and internal scaling in a fixed-point implementation [3], [4].

TABLE II
Elliptic Low-Pass Filter Design Parameters

Parameter	Value
Filter type	Elliptic IIR low-pass
DF1 single biquad order	2
DF1/TDF2 cascade order	4
Sampling frequency f_s	35 511 Hz
Cutoff frequency f_c	200 Hz
Passband ripple R_p	0.1 dB
Stopband attenuation R_s	40 dB
Coefficient format	Q2.28 fixed-point

The passband ripple factor is determined from the specified passband ripple R_p as

$$\epsilon = \sqrt{10^{R_p/10} - 1}. \tag{2}$$

For $R_p = 0.1$ dB, the ripple factor becomes

$$\epsilon = \sqrt{10^{0.1/10} - 1} \approx 0.1526. \tag{3}$$

This small value indicates that the passband amplitude variation is limited, which is important for preserving the desired low-frequency component.

The normalized cutoff frequency used in MATLAB is

$$W_c = \frac{f_c}{f_s/2} = \frac{200}{35\,511/2} \approx 0.01127. \tag{4}$$

Because the cutoff frequency is much lower than the Nyquist frequency, the poles of the designed IIR filter are located close to the unit circle. This makes the filter sensitive to coefficient quantization, especially in the feedback coefficients a_1 and a_2.

At frequencies that lie in the specified stopband, the attenuation requirement can be expressed in linear magnitude form as

$$|H_{\text{stop}}| \le 10^{-R_s/20}. \tag{5}$$

With $R_s = 40$ dB,

$$|H_{\text{stop}}| \le 10^{-40/20} = 0.01. \tag{6}$$

Therefore, once the response has entered the stopband, unwanted high-frequency components should be attenuated to at most 1% of their original amplitude. The 200 Hz value is the passband edge used in the MATLAB design; it should not be interpreted as the beginning of the full 40 dB stopband.

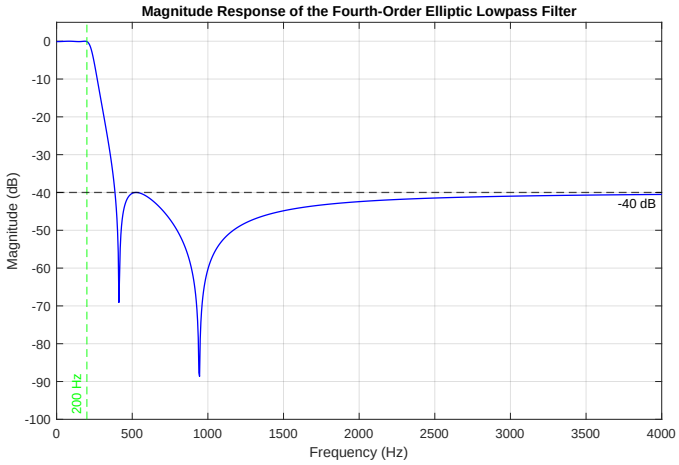


Fig. 2. Magnitude response of the designed fourth-order elliptic lowpass filter ($f_c = 200$ Hz, $R_p = 0.1$ dB, $R_s = 40$ dB).

Fig. 2 shows the magnitude response of the designed fourth-order elliptic lowpass filter. The response contains a low-ripple passband, a steep transition region near the cutoff frequency, and a stopband that satisfies the 40 dB attenuation requirement.

The corresponding pole–zero distribution is shown in Fig. 3. The fourth-order filter contains two complex-conjugate pole pairs and two complex-conjugate zero pairs. All poles remain inside the unit circle, confirming that the floating-point design is stable before coefficient quantization. The pole magnitudes are approximately 0.9765 and 0.9925, so the second pole pair is located very close to the unit circle. This explains the steep transition response, but it also makes the feedback coefficients sensitive to fixed-point rounding.

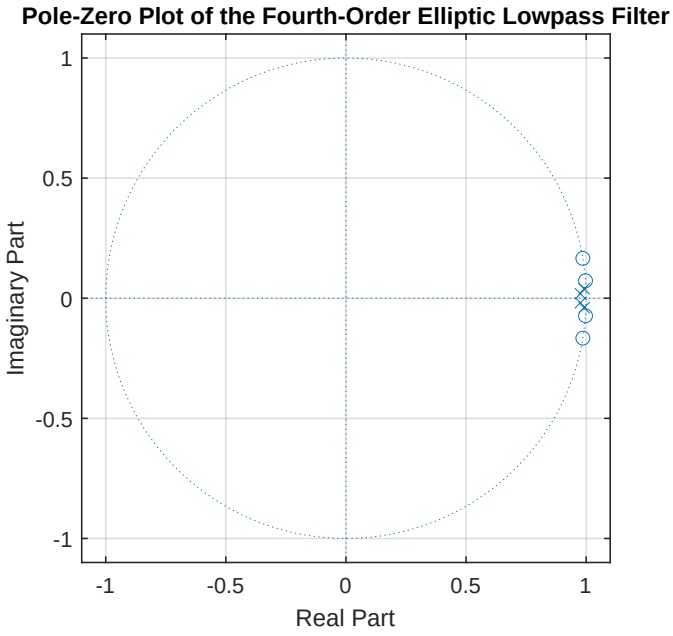


Fig. 3. Pole–zero distribution of the designed fourth-order elliptic lowpass filter.

The zeros lie on the unit circle and create deep attenuation notches in the stopband. After the coefficients are converted to Q2.28, the pole locations must therefore be checked again to ensure that quantization does not move any pole outside the unit circle. The high fractional precision of Q2.28 is selected to preserve these pole locations while still allowing efficient signed fixed-point implementation on the FPGA.

C. Second-Order Section Decomposition and Q2.28 Coefficients

The fourth-order architectures are constructed by cascading two second-order sections (SOSs). Since each section follows the same coefficient representation as the second-order DF1 implementation, the quantization procedure can first be illustrated using the single-biquad coefficients before presenting the complete fourth-order SOS coefficients. The fourth-order transfer function is implemented as the product of two second-order sections,

$$H(z) = H_1(z)H_2(z) = \prod_{i=1}^{2} \frac{b_{0,i} + b_{1,i}z^{-1} + b_{2,i}z^{-2}}{1 + a_{1,i}z^{-1} + a_{2,i}z^{-2}}. \tag{7}$$

Each SOS contains one complex-conjugate pole pair and one complex-conjugate zero pair. This decomposition is preferred over a single fourth-order direct form because it limits

polynomial-coefficient sensitivity and allows the internal signal range to be controlled through section ordering and scaling.

The floating-point coefficient C is mapped to the signed Q2.28 integer C_Q and reconstructed as $\widehat{C}$ according to

$$\begin{aligned} C_Q &= \mathrm{round}\big(C\,2^{28}\big)\,, \\ \widehat{C} &= C_Q\,2^{-28}. \end{aligned} \tag{8}$$

Table IV presents the real-valued coefficients together with the corresponding integers stored in Verilog. Listing one coefficient per row makes the conversion explicit while keeping the table readable within a single IEEE column.

TABLE III
REAL-VALUED AND Q2.28 COEFFICIENTS OF THE SECOND-ORDER DF1 SINGLE BIQUAD

Coeff.	Real value $\widehat{C}$	Q2.28 integer C_Q
b_0	0.010574710	2838627
b_1	-0.017208662	-4619415
b_2	0.010574710	2838627
a_1	-1.915910676	-514298356
a_2	0.919897065	246932988

TABLE IV
REAL-VALUED AND Q2.28 COEFFICIENTS OF THE FOURTH-ORDER SOS FILTER

Stage	Coeff.	Real value $\widehat{C}$	Q2.28 integer C_Q
1	b_0	0.009769134	2622382
1	b_1	-0.019266967	-5171937
1	b_2	0.009769134	2622382
1	a_1	-1.952564932	-524137658
1	a_2	0.953478478	255947430
2	b_0	1.000000000	268435456
2	b_1	-1.994659830	-535437421
2	b_2	1.000000000	268435456
2	a_1	-1.983478274	-532435895
2	a_2	0.985082544	264431082

For example, the Stage-1 feedback coefficient is represented as

$$\mathrm{round}\big(-1.952564932 \times 2^{28}\big) = -524137658. \tag{9}$$

The overall gain returned by the MATLAB SOS decomposition is absorbed into Stage 1, which explains its comparatively small feedforward coefficients. Stage 2 is normalized with $b_0 = b_2 \approx 2^{28}$, but it still contains a complex-conjugate zero pair because b_1 is nonzero. Thus, Stage 2 is not an all-pole section. The selected ordering and scaling keep the intermediate signal level within a practical range while preserving the complete fourth-order transfer function.

For completeness, Table III lists the coefficients used by the second-order DF1 single-biquad reference implementation. The same Q2.28 quantization rule in (8) is applied.

D. DF1 Single Biquad Architecture

The DF1 single biquad implements one second-order IIR section directly from the difference equation obtained by taking the inverse z-transform of the SOS transfer function $H_i(z)$ defined in (7). This canonical realization, in which the feedforward and feedback terms are implemented with separate input and output delay lines, is the standard Direct Form I structure described in classical digital filter references such as [1]. It follows directly from (7): writing one SOS section as

$$H_i(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}} \tag{10}$$

and cross-multiplying gives

$$Y(z)\left(1 + a_1 z^{-1} + a_2 z^{-2}\right) = X(z)\left(b_0 + b_1 z^{-1} + b_2 z^{-2}\right). \tag{11}$$

Applying the inverse z-transform term by term, and using the time-shift property $z^{-k}X(z) \leftrightarrow x[n-k]$ (and likewise for $Y(z)$), yields the difference equation:

$$\begin{aligned} y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] \\ - a_1 y[n-1] - a_2 y[n-2]. \end{aligned} \tag{12}$$

This structure uses separate delay lines for the input and output samples, namely $x[n-1]$, $x[n-2]$, $y[n-1]$, and $y[n-2]$. Its main advantage is structural simplicity and direct correspondence with the mathematical difference equation. Since this realization is second order, it is not directly equivalent in order to the two fourth-order cascade architectures, but it provides a useful reference for resource, timing, and attenuation-slope comparison.

At the input, the 12-bit unsigned ADC value is converted into a signed sample by subtracting the mid-scale offset 2048. The signed sample is then shifted left by 28 bits to enter the Q2.28 fixed-point data path. Inside the biquad, five products are accumulated using an 84-bit signed sum. The result is arithmetically shifted right by 28 bits and then converted back to a 12-bit unsigned output using offset restoration and saturation.

From a hardware scheduling perspective, the four delayed samples are updated only when a new ADC sample is declared valid. During one filter transaction, the current input and the previously stored input/output samples are first sign-extended and used to form the complete multiply–accumulate result. The state registers are updated only after the new output has been calculated, ensuring that the feedback terms always use $y[n-1]$ and $y[n-2]$ rather than partially updated values. Between two valid-sample events, all filter states are held unchanged. This sample-enable organization separates the 50 MHz FPGA clock from the 35 511 Hz signal-sampling rate and prevents the recursive states from being updated more than once for the same ADC sample. Because the single biquad contains no inter-stage register, it also provides a direct basis for comparing arithmetic latency with the two cascaded architectures.

E. DF1 Cascade Architecture

The DF1 cascade architecture connects two DF1 biquad stages in series to realize the fourth-order elliptic filter. Each stage implements (12) with its own SOS coefficient set. The output of Stage 1 is registered before entering Stage 2, as shown in Fig. 5.

The inter-stage pipeline register cuts the long combinational path between the two biquads and allows the synthesis tool to optimize each stage separately. This register introduces one clock-cycle latency between Stage 1 and Stage 2. Since the FPGA clock period is 20 ns and the sampling period is approximately 28.16 μs, this additional latency is negligible for the measured signal. The main benefit is improved timing margin while preserving the fourth-order magnitude response.

F. TDF2 Cascade Architecture

The TDF2 cascade implements the same fourth-order SOS function $H_i(z)$ of (7), but through the TDF2 structure. This realization is obtained by applying the transposition theorem to the canonical Direct Form II (DF2) network of the same SOS section – not to the DF1 network of (12) – a standard derivation presented in [1]. The transposition theorem states that reversing every signal-flow direction in a linear signal-flow graph, exchanging summing junctions with branch (pick-off) points, and swapping the input and output nodes leaves the transfer function unchanged. The DF2 network already merges the feedforward and feedback sums onto a single pair of delay elements; applying the transposition procedure to this network moves the summation points so that each delay element instead accumulates a running partial sum of the feedforward and feedback contributions, denoted here by the internal states $d_1[n]$ and $d_2[n]$. This yields the following internal-state equations

$$\begin{aligned} y[n] &= b_0 x[n] + d_1[n-1], \\ d_1[n] &= b_1 x[n] - a_1 y[n] + d_2[n-1], \\ d_2[n] &= b_2 x[n] - a_2 y[n]. \end{aligned} \tag{13}$$

Compared with DF1, the TDF2 structure uses fewer delay elements per biquad because the feedforward and feedback paths share the internal state variables d_1 and d_2. However, these internal states may have a larger dynamic range than the input and output samples. Therefore, wider 84-bit internal state registers are used to preserve precision and reduce the risk of overflow or limit-cycle behavior.

Although TDF2 uses fewer state variables than DF1, its wide internal states can still increase flip-flop utilization. The three data paths are collected in Figs. 4–6 for a direct structural comparison.

G. Fixed-Point Quantization and Overflow Handling

The filter uses Q2.28 coefficients and wider signed internal signals. Under the Q-format convention adopted in this work, the two bits to the left of the binary point include the sign bit; therefore, Q2.28 represents values in $[-2,\, 2 - 2^{-28}]$. The coefficients are stored in 32-bit signed Verilog objects, with sign extension applied to the effective Q2.28 value. This format is selected because the second-SOS feedback coefficient a_1 has a magnitude close to two. A Q1.14 representation under the same convention has range $[-1,\, 1 - 2^{-14}]$ and cannot represent that coefficient, whereas Q2.28 provides both the required range and a resolution of 2^{-28}.

Overflow risk exists at three locations: offset conversion at the input, multiply-accumulate operations inside the filter, and output conversion back to the 12-bit ADC range. The input conversion uses signed extension after subtracting the mid-scale offset. The accumulator width is selected as 84 bits because a 52-bit data value multiplied by a 32-bit coefficient can produce an 84-bit product. After accumulation, the result is shifted right by 28 bits using arithmetic shifting so that the sign is preserved. After arithmetic right shifting, the centered result is converted back to the unsigned ADC domain as

$$u[n] = \mathrm{round}\left(\frac{y_Q[n]}{2^{28}}\right) + 2048. \tag{14}$$

Saturation then clamps the restored value to the valid 12-bit range:

$$y_{\mathrm{out}}[n] = \begin{cases} 0, & u[n] < 0, \\ u[n], & 0 \le u[n] \le 4095, \\ 4095, & u[n] > 4095. \end{cases} \tag{15}$$

Saturation is preferred over wrap-around because wrap-around can produce a sudden sign inversion and excite the IIR feedback loop.

A warm-up counter is used at startup. During the first 256 valid samples, the output is held at the mid-scale value. This avoids reporting the initial transient response caused by zero-initialized feedback states as a valid steady-state output. After the warm-up period, the output follows the computed filter response.

This warm-up interval must be distinguished from two other time scales used elsewhere in this paper. The first is the processing latency reported in Section III-D3, which is only the fixed number of enable cycles (1–2 cycles, 28.16–56.32 μs) required for one valid ADC sample to propagate through the multiply–accumulate datapath; it reflects arithmetic depth, not filter dynamics. The second is the transient-decay, or settling, time of the recursive feedback loop itself, which is governed by the dominant pole radius rather than by arithmetic depth. For a pole of magnitude r close to the unit circle, the sample index at which the free response decays to a fraction δ of its initial value satisfies

$$n_\delta = \frac{\ln \delta}{\ln r}. \tag{16}$$

Using the dominant pole magnitude $r \approx 0.9925$ identified in Section II-B, the time constant is $\tau = -1/\ln r \approx 133$ samples (≈ 3.74 ms at $f_s = 35\,511$ Hz), and decay to within 1% of the initial transient requires $n_{1\%} \approx 612$ samples (≈ 17.2 ms). The 256-sample warm-up interval therefore corresponds to a residual transient of about $r^{256} \approx 14.6\%$ of its initial amplitude; it is intended only as a conservative guard against the largest, most disruptive part of the start-up transient (e.g., a saturation event at the output), not as a guarantee of full settling. Full settling for the frequency-domain results reported later in this paper is instead ensured by a longer, separately chosen 50 ms discard interval.

H. Implementation Summary

Table V summarizes the key structural features of the three implemented architectures. This table separates the algorithmic

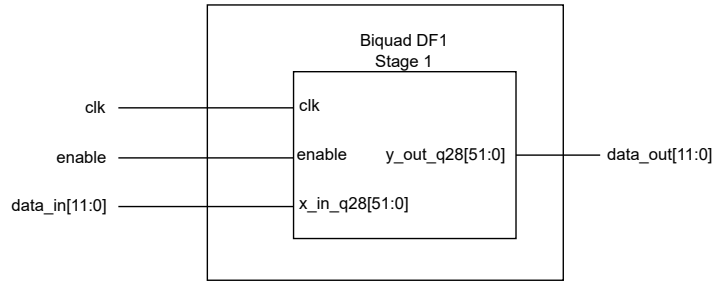


Fig. 4. DF1 single biquad data path showing separate input and output delay registers.

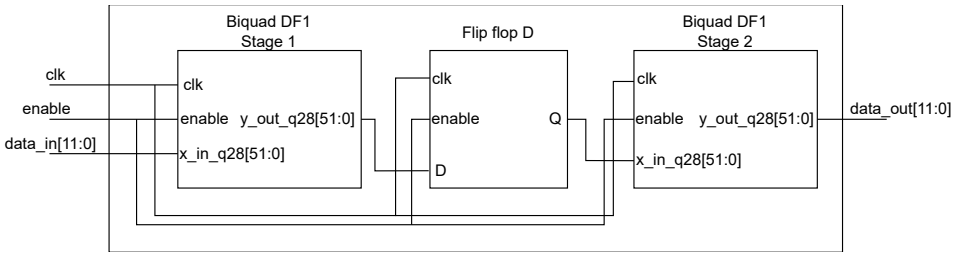


Fig. 5. DF1 cascade architecture with two biquad sections and an inter-stage pipeline register.

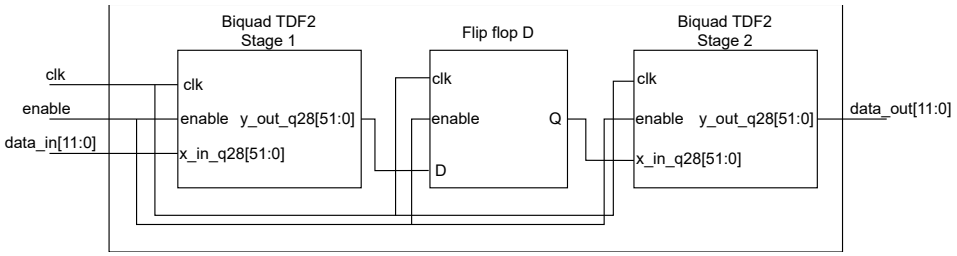


Fig. 6. TDF2 cascade architecture using two transposed biquad sections and wide internal state registers.

difference from the measured resource and timing results, which are presented later in Section III.

TABLE V
ARCHITECTURAL COMPARISON OF THE FPGA IIR IMPLEMENTATIONS

Feature	DF1 Biquad	DF1 Cascade	TDF2 Cascade
Filter order	2	4	4
SOS stages	1	2	2
Total states	4	8	4
Inter-stage pipeline	No	Yes	Yes
Accumulator width	84 bit	84 bit	84 bit
Coefficient format	Q2.28	Q2.28	Q2.28
Output protection	Saturation	Saturation	Saturation
Main advantage	Simplicity	Timing balance	Fewer DSPs
Main drawback	Lower order	More DSPs	Wider states

III. EXPERIMENTAL RESULTS AND DISCUSSION

A. Hardware Verification Setup

The hardware verification model consists of the MCP4921 signal generation block, the MCP3208 ADC acquisition block, the FPGA filtering block, and the UART interface to the host PC. The main comparison uses two single-tone test conditions. The 150 Hz case is selected inside the passband to verify that the desired low-frequency component is preserved. The 250 Hz case is selected above the 200 Hz cutoff frequency and near the transition region, where the magnitude response changes rapidly. This case is more informative than a deeply passband or deeply stopband signal because it reveals practical attenuation differences between the MATLAB reference and the fixed-point FPGA implementation. A 1.65 V DC input is also used to evaluate steady-state stability and possible limit-cycle oscillation in the IIR feedback loop.

The physical test bench is shown in Fig. 7. The setup includes the EBAZ4205 board, the MCP4921/MCP3208 interface circuitry, the ESP32 waveform generator, a CP2102 USB-to-UART bridge for transferring the filtered output samples from the FPGA to the host computer, and an FT232H interface used separately for programming the FPGA bitstream from Vivado. During measurement, the analog test signal is sampled by the MCP3208, processed by the selected FPGA filter, and the filtered 12-bit results are transmitted to the computer through the CP2102 UART data path. The FT232H interface is used only for FPGA configuration and does not participate in the measured data-acquisition path. The UART block sends one 12-bit result every four ADC conversions, packaged as two bytes; therefore, the host-side capture rate is approximately $f_s/4 = 8\,878$ sample/s. Python records the serial data, and MATLAB displays the waveform and computes the one-sided FFT. The photograph therefore documents the complete end-to-end verification path used to obtain the experimental results

rather than a simulation-only arrangement.

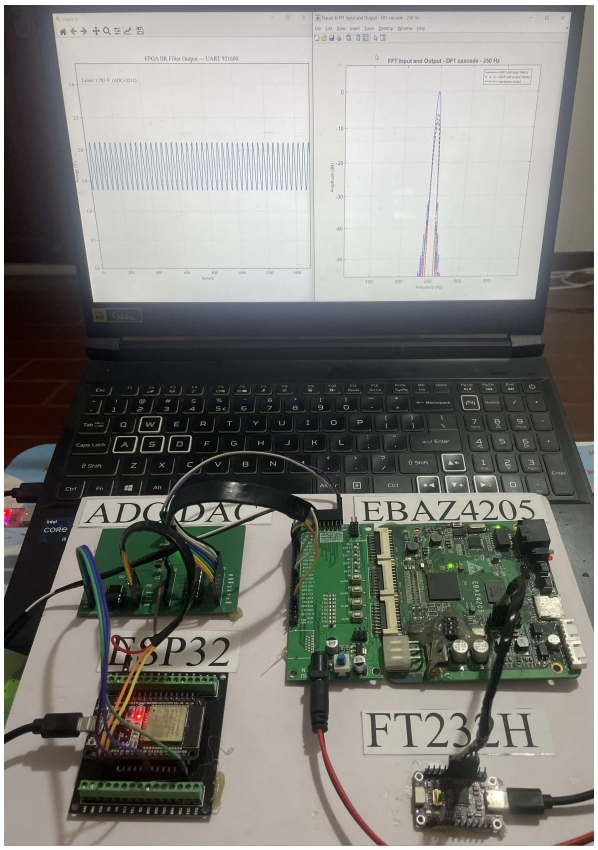


Fig. 7. Physical hardware verification setup comprising the ADC/DAC interface, EBAZ4205 FPGA board, ESP32 waveform generator, CP2102 USB-to-UART bridge for filtered-data acquisition, FT232H interface for Vivado bitstream programming, and host-side waveform and FFT display.

Table VI summarizes the numerical and interface configuration used for the measurements.

TABLE VI
HARDWARE VERIFICATION CONFIGURATION

Parameter	Value
FPGA board	EBAZ4205 (Zynq XC7Z010)
System clock	50 MHz
Design sampling rate f_s	35 511 Hz
Fixed-point format	Q2.28
DAC	MCP4921, 12-bit SPI [9]
ADC	MCP3208, 12-bit SPI [8]
FPGA programming interface	FT232H
UART data interface	CP2102 USB-to-UART bridge
UART baud rate	921 600 baud
Host capture rate	$f_s/4 \approx 8878$ sample/s

B. Single-Tone Filtering Accuracy

As outlined in Section I, the single-tone evaluation focuses on 150 Hz and 250 Hz around the 200 Hz cutoff, with an additional 350 Hz case for the DF1 single biquad only. The measured waveforms and spectra for all three architectures are presented first, followed by the detailed rationale for each test frequency and the quantitative dB comparison in Table VII.

1) Hardware Output Waveforms and FFT Spectra: Before presenting the figures, the main trends of the measured results can be summarized as follows. For the DF1 single-biquad, the 150 Hz waveform remains close to the input amplitude because this frequency lies inside the passband. At 250 Hz, the output begins to decrease because the excitation is already above the 200 Hz cutoff. The additional 350 Hz result shows a further reduction in amplitude, but the attenuation slope is still gradual because this implementation is only second order.

For the fourth-order DF1 cascade, the 150 Hz component is also preserved, whereas the 250 Hz component is attenuated much more strongly than in the single-biquad case. This behavior is consistent with the steeper transition band of the fourth-order elliptic response. The TDF2 cascade shows the same overall frequency-selective behavior as the DF1 cascade because both architectures implement the same two SOS sections using the same Q2.28 coefficients. Their main differences therefore appear in internal state organization, FPGA resource usage, and timing rather than in the ideal transfer function.

Figs. 8, 9, and 10 present all time-domain and frequency-domain results before the quantitative discussion. In every row, the left panel is the UART-captured hardware waveform and the right panel is the corresponding one-sided FFT comparison with the MATLAB reference. The two panels use the same visual width so that the time-domain and frequency-domain results are balanced on the page.

All single-tone measurements are performed with the same acquisition chain and processing settings. The DAC amplitude, ADC reference voltage, sampling frequency, warm-up interval, number of recorded samples, FFT window, and amplitude-normalization procedure are kept unchanged between architectures. Consequently, the measured differences mainly reflect the implemented filter response rather than changes in the experimental configuration. A 50 ms interval ($f_s \times 50$ ms ≈ 1776 samples) is discarded before spectral analysis, which exceeds the dominant-pole settling estimate of $n_{1\%} \approx 612$ samples (≈ 17.2 ms) derived in Section II-G by roughly a factor of three; therefore, the reported amplitudes represent steady-state behavior. This 50 ms interval should not be confused with the much shorter 256-sample warm-up counter, which only guards against the most disruptive part of the start-up transient, nor with the microsecond-scale arithmetic pipeline latency reported in Section III-D3.

Beyond confirming the expected passband/transition-band trend already motivated in Section III-A, the 150 Hz agreement across all three architectures also confirms that the ADC centering, Q2.28 scaling, offset restoration, and UART reconstruction are operating consistently. The corresponding 250 Hz and 350 Hz deviations are examined quantitatively together with Table VII below.

2) FFT-Based Frequency-Domain Verification: The FFT results are used to quantify the attenuation level in the frequency domain, summarized in Table VII. At 150 Hz, all three architectures show a hardware-to-MATLAB deviation below 0.11 dB, confirming that the desired low-frequency

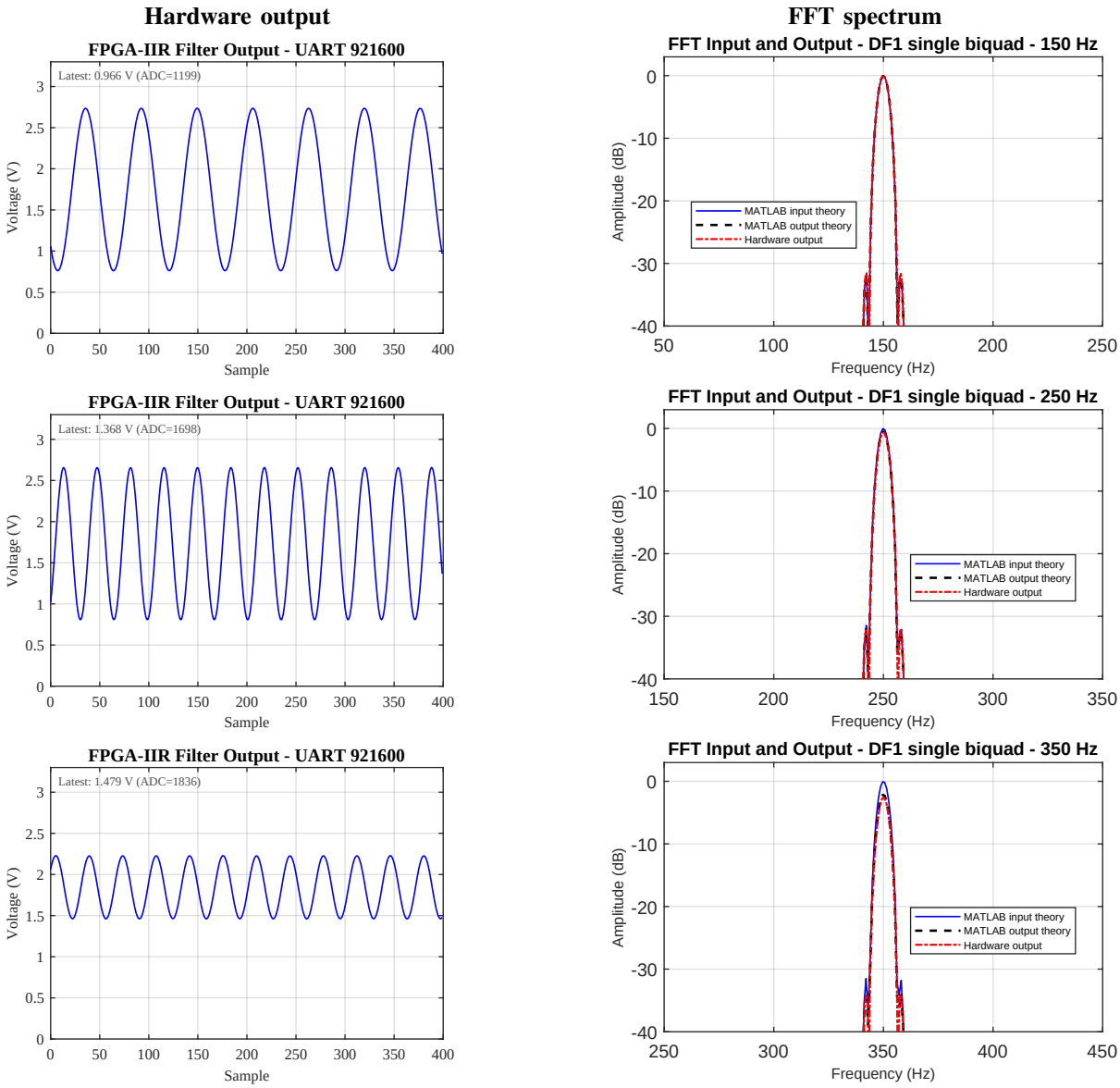


Fig. 8. Side-by-side hardware output and FFT verification for the DF1 single biquad. The 350 Hz case is included only for this second-order to illustrate its gradual attenuation slope beyond the cutoff frequency.

component is preserved after Q2.28 quantization and FPGA implementation. At 250 Hz, small differences associated with ADC/DAC nonidealities, finite word length, sampling-rate estimation, and spectral leakage result in a larger dB deviation; the fourth-order cascade architectures show much stronger attenuation than the DF1 single biquad, which agrees with the steeper roll-off of the fourth-order elliptic filter.

Table VII confirms, with actual dB figures, the near-identical DF1-cascade and TDF2-cascade attenuation already noted qualitatively above: the magnitude response is governed primarily by filter order and coefficient quantization rather than by the DF1/TDF2 internal realization.

For the additional 350 Hz observation, the DF1 single biquad produces a MATLAB attenuation of -2.100 dB and a measured hardware attenuation of -2.615 dB. Compared with the 250 Hz result, the attenuation increases as the input frequency moves farther from the cutoff frequency. However, the attenuation slope remains relatively gradual because this architecture is only a second-order section. This additional observation supports the conclusion that the DF1 single biquad is simpler and less resource-demanding, but its transition-band selectivity is weaker

TABLE VII
SINGLE-TONE FILTERING ACCURACY AT 150 HZ AND 250 HZ

Architecture	f (Hz)	MATLAB (dB)	Hardware (dB)	Error (dB)
DF1 single biquad	150	-0.002	-0.105	-0.104
	250	-0.437	-0.670	-0.234
	350	-2.100	-2.615	-0.515
DF1 cascade	150	-0.099	-0.191	-0.092
	250	-5.955	-8.335	-2.379
TDF2 cascade	150	-0.099	-0.193	-0.093
	250	-5.955	-8.336	-2.381

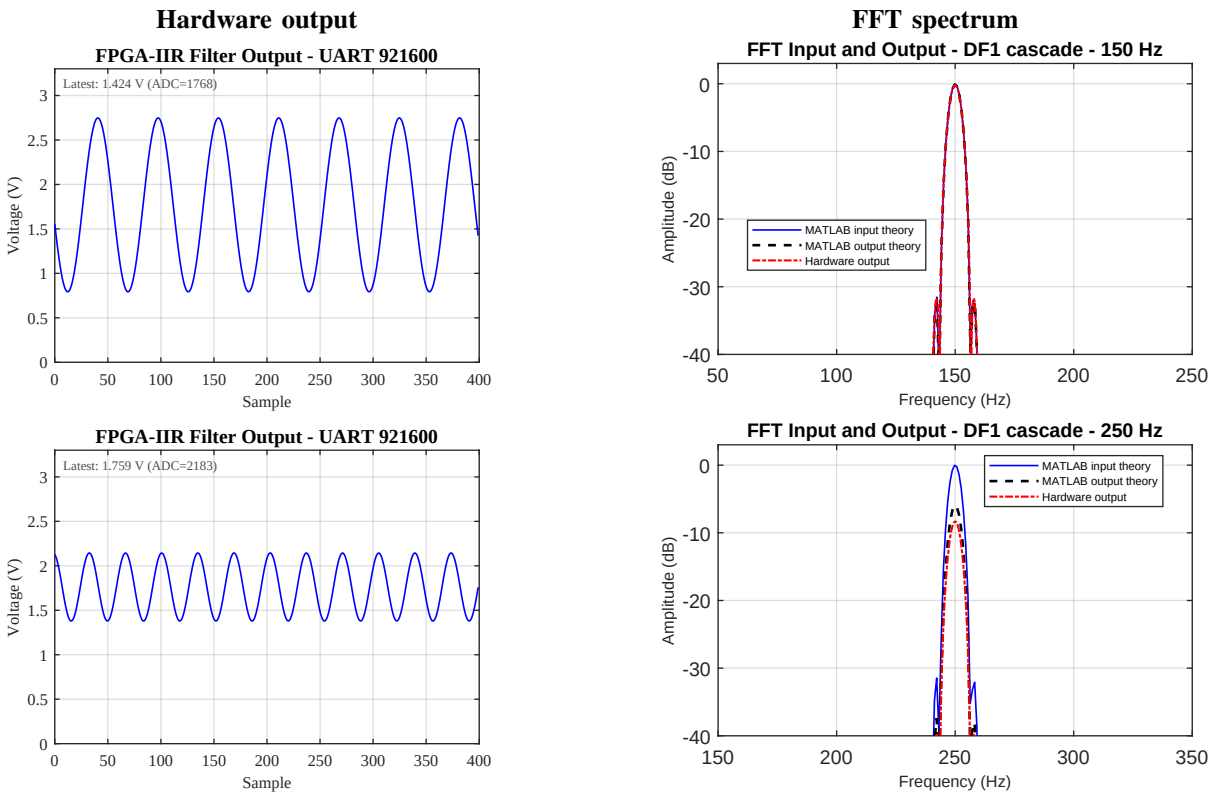


Fig. 9. Side-by-side hardware output and FFT verification for the fourth-order DF1 cascade architecture at 150 Hz and 250 Hz.

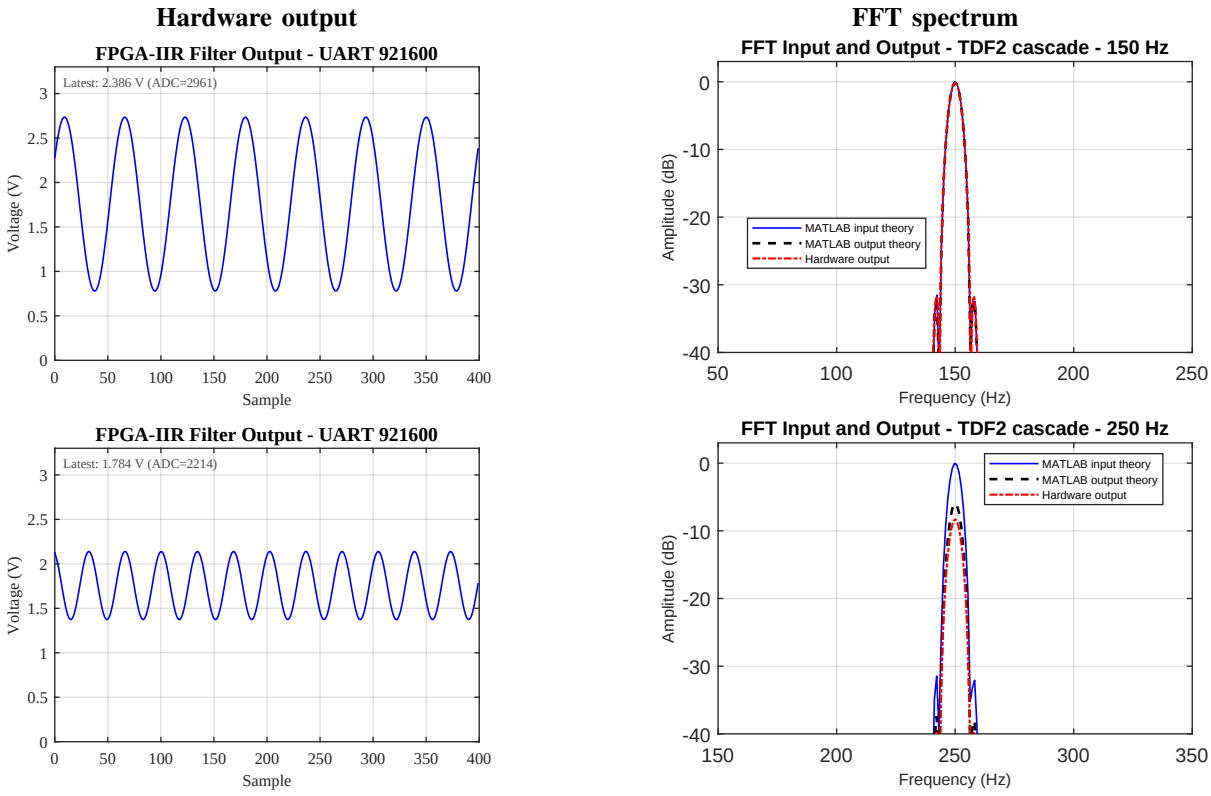


Fig. 10. Side-by-side hardware output and FFT verification for the fourth-order TDF2 cascade architecture at 150 Hz and 250 Hz.

than the fourth-order cascade architectures.

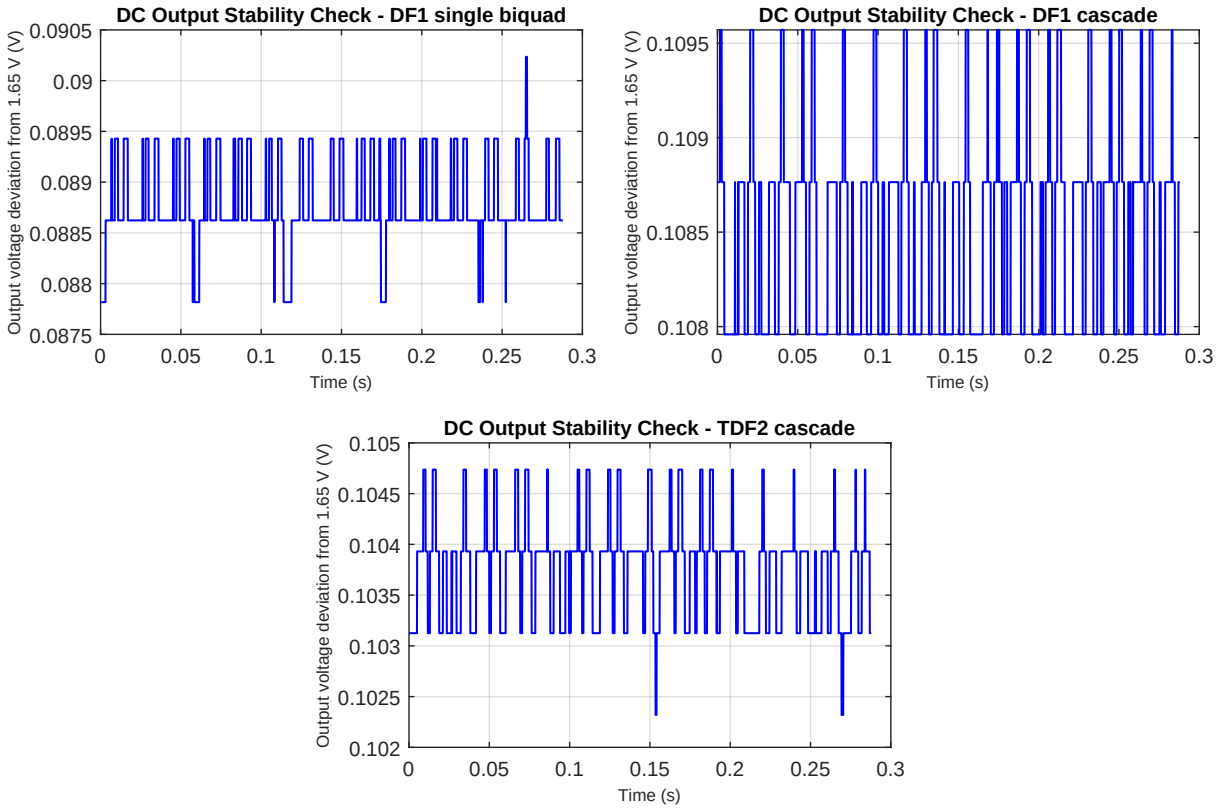


Fig. 11. Enlarged DC stability waveforms for the three architectures.

C. DC Stability and Limit-Cycle Observation

A 1.65 V DC input is applied to evaluate steady-state stability and to check whether the recursive fixed-point implementation produces a visible limit-cycle oscillation. Since the ADC samples are centered by subtracting the mid-scale code before filtering, the reported mean values represent the residual output relative to the 1.65 V reference rather than the absolute DC level.

For an ideal mid-scale DC input, subtracting the ADC offset produces a centered input close to zero. A numerically stable IIR implementation should therefore converge to a nearly constant residual after the startup transient has disappeared. Any nonzero average level can be attributed mainly to DAC/ADC offset, coefficient quantization, and rounding in the fixed-point data path. In contrast, a persistent periodic variation under a constant input would indicate a possible zero-input limit cycle generated by the recursive feedback states.

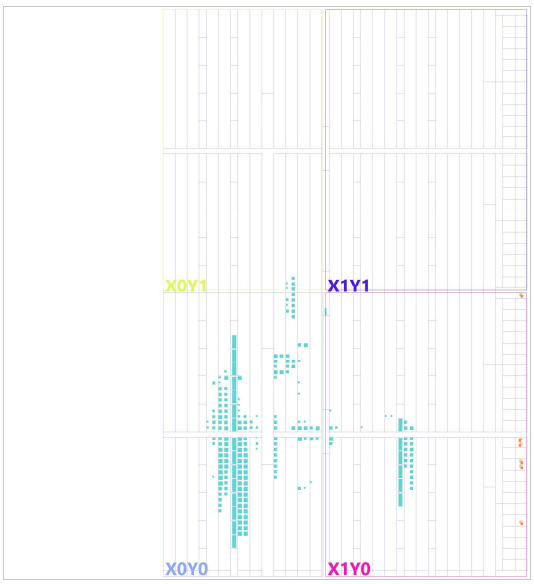
The DC records are evaluated only after the warm-up interval so that the initial response of the zero-initialized delay registers does not affect the steady-state statistics. For each architecture, the mean residual is used to measure the remaining DC offset, while the standard deviation describes the short-term fluctuation around that mean. Since all three implementations use the same signal source, ADC, sampling frequency, and UART acquisition path, differences among the measured traces primarily reflect filter order, internal state organization, and fixed-point rounding behavior.

The DC plots should be interpreted differently from the sinusoidal waveforms. Because the signal is reconstructed from 12-bit converter codes, a stable output may appear as a set of closely spaced horizontal levels rather than a perfectly smooth line. Small transitions between adjacent levels mainly represent converter quantization and fixed-point rounding. A limit cycle would instead appear as a persistent, structured oscillation under a constant centered input after the startup transient has decayed. Accordingly, the visual inspection is combined with the measured mean residual and standard deviation rather than relying on waveform appearance alone.

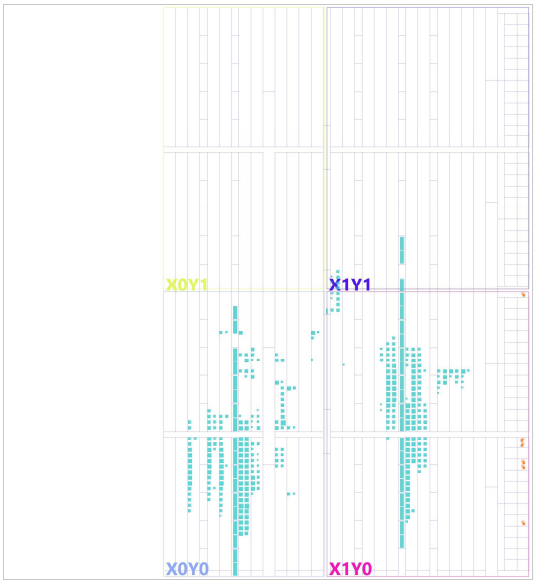
TABLE VIII
DC STABILITY: RESIDUAL OUTPUT MEAN AND STANDARD DEVIATION

Architecture	Mean residual (V)	Std. deviation (V)
DF1 single biquad	0.088776	0.000419
DF1 cascade	0.108515	0.000546
TDF2 cascade	0.103751	0.000516

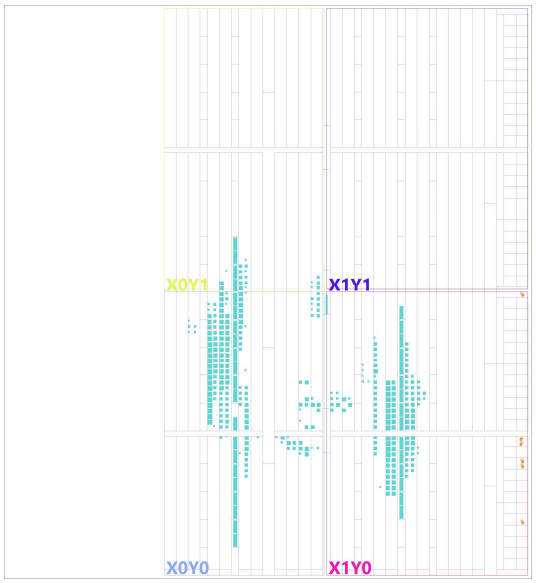
The measured residual offsets of the DF1 and TDF2 cascades are larger than that of the DF1 single biquad. This difference cannot be attributed to filter order alone because the same analog chain also contributes DAC, ADC, and reference-voltage offsets. Nevertheless, the standard deviation remains below one millivolt for all three architectures, and no persistent periodic pattern is visible. These observations indicate stable steady-state operation with no significant limit-cycle activity in the recorded data.



(a) DF1 single biquad



(b) DF1 cascade



(c) TDF2 cascade

Fig. 12. Vivado post-implementation device views for the three filter architectures.

D. FPGA Resource Utilization and Timing

All three architectures are synthesized, placed, and routed separately on the same Zynq XC7Z010 device using Vivado with an identical 50 MHz clock constraint. For each implementation, only one filter architecture is enabled in the top-level module.

Fig. 12 shows the resulting post-implementation device views for the three architectures. These layouts are useful for identifying the physical spread of the design and possible localized congestion, but they should not be used as a substitute for the utilization report. A visually larger cluster does not necessarily imply a proportional increase in LUT, flip-flop, or DSP count because different primitive types occupy different FPGA columns. The quantitative comparison is therefore made using the post- implementation values reported immediately after the figure.

1) Resource Utilization: Fig. 13 presents the same results graphically, using the original resource counts rather than percentages. In the chart, the resource groups are ordered as DSPs, FFs, slices, and LUTs so that the legend does not obscure the tallest LUT bars.

The DF1 single biquad uses 492 LUTs, 10 flip-flops, 147 slices, and 21 DSP blocks. Its lower resource usage is expected because it contains only one second-order section rather than an equal-order implementation.

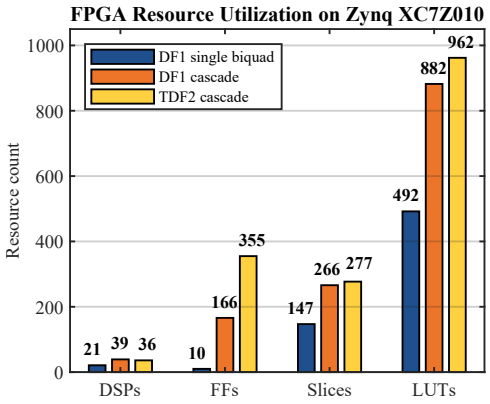


Fig. 13. Post-implementation FPGA resource counts for the three filter architectures. The resource groups are ordered as DSPs, FFs, slices, and LUTs.

The fourth-order DF1 cascade uses 882 LUTs, 166 flip-flops, 266 slices, and 39 DSP blocks. The increase relative to the single biquad is mainly caused by the second SOS stage, the duplicated input/output delay lines, and the inter-stage register used to separate the two arithmetic paths.

The TDF2 cascade uses 962 LUTs, 355 flip-flops, 277 slices, and 36 DSP blocks. Compared with the DF1 cascade, TDF2 saves three DSP blocks, but it requires 80 additional LUTs, 189 additional flip-flops, and 11 additional slices. Although TDF2 uses fewer state variables per biquad, its internal accumulated states are wider, which increases the register and logic cost.

2) Timing Closure: Timing is evaluated from the post-route Vivado timing summary. Since the required clock period is 20 ns, a positive Worst Negative Slack indicates that the longest register-to-register path is shorter than the specified period. A first-order slack-derived frequency estimate is obtained from $F_{\max,\mathrm{est}} = 1/(20\ \mathrm{ns} - \mathrm{WNS})$. This estimate does not replace a separate implementation run with a tighter clock constraint.

All three implementations meet the 50 MHz timing constraint. The DF1 cascade provides the largest positive slack and the highest slack-derived estimate, approximately 50.12 MHz. Although it contains two biquads, the pipeline register between the stages prevents the two multiply–accumulate networks from forming one long combinational path. The TDF2 cascade also meets timing, with a slack-derived estimate of approximately 50.10 MHz. The single biquad gives approximately 50.05 MHz.

Although the three architectures exhibit different arithmetic organizations, the physical implementation confirms that all designs remain compact on the XC7Z010 device. The implementation reports also indicate that routing congestion is limited and that the placement density remains well distributed across the programmable logic. These observations motivate a quantitative comparison using the post-route timing report and the measured processing latency discussed in the following subsections.

The small difference among the three slack-derived frequency estimates indicates that the 50 MHz clock constraint is close to the achieved post-route limit. Nevertheless, the filter is enabled only once for each ADC sample at approximately 35 511 Hz. Thus, the arithmetic latency of one or two 50 MHz clock cycles is negligible compared with the sampling interval. The positive slack values confirm that the implemented filters complete their registered computations before the next active clock edge and can operate without timing violations under the selected constraint.

TABLE IX
TIMING SUMMARY AT 50 MHZ CLOCK CONSTRAINT

Architecture	WNS (ns)	$F_{\max,\mathrm{est}}$ (MHz)	Timing
DF1 single biquad	+0.018	50.05	Met
DF1 cascade	+0.047	50.12	Met
TDF2 cascade	+0.038	50.10	Met

3) Processing Latency: Beyond the post-route timing slack, the end-to-end processing latency of each architecture is also relevant for real-time operation. Latency is defined here as the number of enable cycles required from a valid ADC sample entering the filter core until the corresponding filtered output becomes stable. Since the filter core is updated only once per valid ADC sample, one enable cycle corresponds to $50 \times 10^6/35\,511 \approx 1408$ system clock cycles.

TABLE X
PROCESSING LATENCY OF THE THREE FILTER ARCHITECTURES

Architecture	Enable cycles	Clock cycles	Latency (µs)
DF1 single biquad	1	1408	28.16
DF1 cascade	2	2816	56.32
TDF2 cascade	2	2816	56.32

For the DF1 single biquad, the entire multiply-accumulate computation is implemented as purely combinational logic with no intermediate pipeline register; the output becomes valid within the same enable cycle in which the input is presented, giving a latency of 1 enable cycle (1408 clock cycles, 28.16 µs). For the DF1 cascade and TDF2 cascade, the inter-stage pipeline register introduced for timing closure (Section II-E) delays Stage 2's input by one additional enable cycle relative to Stage 1, so the final output stabilizes after 2 enable cycles (2816 clock cycles, 56.32 µs), as summarized in Table X.

The maximum latency among the three architectures (56.32 µs) is several orders of magnitude smaller than the period of the lowest test frequency used in this study (50 Hz, $T = 20$ ms). This confirms that the pipeline register inserted for timing closure has no practical impact on filtering quality within the tested frequency range.

IV. CONCLUSION

This paper presented the design and FPGA implementation of elliptic IIR low-pass filter architectures on the EBAZ4205 Zynq XC7Z010 platform. The hardware evaluation focused on single-tone inputs at 150 Hz and 250 Hz, which examine passband preservation and transition-band attenuation near the 200 Hz cutoff frequency. The 350 Hz case was included only as an additional DF1 observation to show the gradual attenuation slope of the second-order. The DC test confirmed stable steady-state operation with no significant limit-cycle activity. Resource

and timing comparison showed that no single architecture is optimal for every metric. The DF1 cascade provides the largest positive timing slack and a balanced resource profile, while the TDF2 cascade saves 7.69% DSP slices compared with DF1 cascade but increases LUT, FF, and slice utilization. These results provide practical guidance for selecting an IIR filter architecture under fixed-point resource constraints on low-cost FPGA platforms. Future work will extend the design to higher sampling rates, explore higher-order and adaptive filter structures, and investigate automated fixed-point coefficient and Verilog code generation.

ACKNOWLEDGMENT

The authors would like to thank the Faculty of Electrical and Electronics Engineering, Ho Chi Minh City University of Technology and Education, for supporting the implementation and verification of this project.

REFERENCES

[1] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing*, 4th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2006.

[2] G. A. Constantinides, P. Y. K. Cheung, and W. Luk, *Synthesis and Optimization of DSP Algorithms*. Dordrecht, The Netherlands: Kluwer Academic Publishers, 2004, doi: 10.1007/b116503, https://dsp-book.narod.ru/SAODSP.pdf

[3] D. Debarshi and S. D. Himadri, "Synthesis and implementation of an IIR filter using Verilog language," *Int. J. Comput. Appl.*, vol. 186, no. 8, pp. 15–21, 2024, https://www.researchgate.net/publication/377114969_Synthesis_and_Implementation_of_an_IIR_Filter_Using_Verilog_Language

[4] J. Carletta, R. J. Veillette, F. W. Krach, and Z. Fang, "Determining appropriate precisions for signals in fixed-point IIR filters," in *Proc. 40th Design Autom. Conf. (DAC)*, Anaheim, CA, USA, Jun. 2003, pp. 656–661, doi: 10.1145/775832.775998, https://www.cecs.uci.edu/~papers/compendium94-03/papers/2003/dac03/pdffiles/38_3.pdf

[5] W. Kester, Ed., *Mixed-Signal and DSP Design Techniques*. Boston, MA, USA: Newnes, 2003, https://pce-fet.com/common/library/books/16/9335_[Walt_Kester__Engineering_Staff__Analog_Devices_(b-ok.org).pdf

[6] S. Saha and S. Barman Mandal, "FPGA implementation of IIR elliptic filters for de-noising ECG signal," *Biomedical Signal Processing and Control*, vol. 96, Art. no. 106544, 2024, doi: 10.1016/j.bspc.2024.106544, https://www.sciencedirect.com/science/article/abs/pii/S1746809424006025

[7] A. Eddla and V. Y. Jayasree Pappu, "Optimal IIR Filter Design and FPGA Realization," *Eng. Technol. Appl. Sci. Res.*, vol. 16, no. 1, pp. 31009–31014, Feb. 2026, doi: 10.48084/etasr.12837, https://etasr.com/index.php/ETASR/article/view/12837

[8] Microchip Technology Inc., *MCP3204/3208: 2.7V 4-Channel/8-Channel 12-Bit A/D Converters with SPI Serial Interface*, DS21298E, 2008, https://ww1.microchip.com/downloads/en/DeviceDoc/21298E.pdf

[9] Microchip Technology Inc., *MCP4921/4922: 12-Bit DAC with SPI Interface*, DS21897A, 2004, https://ww1.microchip.com/downloads/en/DeviceDoc/21897A.pdf

[10] Xilinx Inc., *Zynq-7000 SoC Data Sheet: Overview*, DS190, rev. 1.11.1, Jul. 2018, https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview

[11] G. Tatar, İ. Çiçek, and S. Bayar, "FPGA Design of a Fourth Order Elliptic IIR Band-Pass Filter Using LabVIEW," *European Journal of Science and Technology*, no. 26, pp. 122–127, 2021.

[12] K. Achtenberg, R. Szplet, and Z. Bielecki, "Advanced, Real-Time Programmable FPGA-Based Digital Filtering Unit for IR Detection Modules," *Electronics*, vol. 13, no. 22, Art. no. 4449, Nov. 2024, doi: 10.3390/electronics13224449.

[13] H. Yi, F. Chang, P. Tan, B. Huang, Y. Wu, Z. Xu, B. Ma, and J. Ma, "Design of infinite impulse response maximally flat stable digital filter with low group delay," *Scientific Reports*, vol. 15, no. 1, Art. no. 11074, Apr. 2025, doi: 10.1038/s41598-025-87175-5.

[14] B. Sindhuri Kandula, S. Patchala, K. Surapaneni, M. Aravind Kumar, C. Ram Grandhi, and P. R. Desamala, "FPGA design of an efficient divide-and-conquer multiplier based on multiple stage fast ripple hybrid adders for peak cancellation with IIR filters," *International Journal of Electronics*, vol. 113, no. 1, pp. 109–130, 2025, doi: 10.1080/00207217.2025.2500779.